

LabGym Command Line Interface (CLI) Guide

This guide details the implementation, command syntax, configuration architecture, and parameter mapping of the headless command-line interface (CLI) for LabGym.

1. Architecture & GUI Alignment

The LabGym CLI is fully decoupled from the **wxPython** GUI, allowing training, evaluation, and inference workflows to run headlessly on remote cloud servers or High-Performance Computing (HPC) clusters utilizing schedulers like SLURM.

GUI-to-CLI Parameter Mapping

The CLI options align directly with parameters defined across the three main GUI panels:

GUI Panel Field / Option	CLI Option Flag	YAML Configuration Key	Notes / Default Values
Train Categorizers			
Select folder of sorted examples	<code>--data_path</code>	<code>data_path</code>	Dataset directory path
Select cache folder for prepared examples	<code>--out_folder</code>	<code>out_folder</code>	Temporary directory path
Specify network complexity/type	<code>--network_type</code>	<code>network_type</code>	<code>pattern_recognizer</code> , <code>animation_analyzer</code> , <code>combnet</code>
Complexity levels (Lv1 to Lv5)	<code>--level_conv</code> , <code>--level_tconv</code>	<code>level_conv</code> , <code>level_tconv</code>	Network architectures complexity
Input width, height, channels	<code>--dim_conv</code> , <code>--dim_tconv</code> , <code>--channel</code>	<code>dim_conv</code> , <code>dim_tconv</code> , <code>channel</code>	Resolution constraints (e.g., <code>64</code> , <code>32</code> , <code>1</code>)
Frames per animation sequence	<code>--time_step</code>	<code>time_step</code>	Episode window duration (default <code>15</code>)
Augmentation methods	<code>--aug_methods</code>	<code>aug_methods</code>	List of strings (e.g. <code>['rotate', 'flip']</code>)

GUI Panel Field / Option	CLI Option Flag	YAML Configuration Key	Notes / Default Values
Save folder for training reports	<code>--out_path</code>	<code>out_path</code>	Export logs directory
On-the-fly training	<code>--training_onfly</code>	<code>training_onfly</code>	Avoid caching prepared examples (<code>true/false</code>)
Train Detectors			
Select training images folder	<code>--path_to_trainingimages</code>	<code>path_to_trainingimages</code>	Directory containing raw pictures
Select JSON annotation file	<code>--path_to_annotation</code>	<code>path_to_annotation</code>	COCO-formatted annotations
Inferencing framesize	<code>--inference_size</code>	<code>inference_size</code>	Image frame size (default <code>480</code>)
Training iterations	<code>--iteration_num</code>	<code>iteration_num</code>	Backpropagation cycles (default <code>200</code>)
Analyze Behaviors (Infer)			
Select Categorizer model	<code>--path_to_categorizer</code>	<code>path_to_categorizer</code>	Trained model path
Select videos / images	<code>--path_to_videos</code>	<code>path_to_videos</code>	Input directory or list of file paths
Select output folder	<code>--result_path</code>	<code>result_path</code>	Target output directory
Specify object detector method	<code>--use_detector, --path_to_detector</code>	<code>use_detector, path_to_detector</code>	Toggle and path for Det2 model
Analysis start offset	<code>--t</code>	<code>t</code>	Starting second offset (default <code>0.0</code>)
Analysis duration	<code>--duration</code>	<code>duration</code>	Total evaluation length (default <code>0.0</code> = end)
Number of animals	<code>--animal_number</code>	<code>animal_number</code>	Total target counts (integer or JSON map)
Target behaviors to annotate	<code>--behavior_to_include</code>	<code>behavior_to_include</code>	List of target behavior names (default <code>['all']</code>)

GUI Panel Field / Option	CLI Option Flag	YAML Configuration Key	Notes / Default Values
Target measurements	<code>--parameter_to_analyze</code>	<code>parameter_to_analyze</code>	Kinematic options

2. CLI Command Syntax & Option Overrides

Every parameter mapped in the configuration files is exposed as an explicit CLI option flag. In-line flags dynamically override matching keys passed inside a YAML file.

Default GUI Fallback

If you run `labgym` without any subcommands, it will fall back to starting the desktop GUI interface:

```
labgym
```

Subcommand `train`

Headless training execution for Categorizers (sub-networks) or Detectors (object detection).

- **Syntax:**

```
labgym train --config <file.yaml> [inline-overrides]
```

- **Inline Parameters:**

- `--network_type`: Choice of `pattern_recognizer`, `animation_analyzer`, `combnet`, `detector`
- `--model_path`: Output model folder target
- `--data_path`: Training examples directory path
- `--training_onfly`: Enable/disable on-the-fly training (accepts `true` or `false`)
- `--path_to_annotation`: Path to COCO annotation file (required if `--network_type` `detector`)
- `--path_to_trainingimages`: Path to training images folder (required if `--network_type` `detector`)
- `--iteration_num`: Detector training iterations (integer)
- `--inference_size`: Detector inference frame size (integer)

Subcommand `test`

Headless evaluation of a model against a ground-truth dataset. This subcommand automatically calculates mathematically exact centroids of behaviors in the high-dimensional penultimate-layer feature space using **Cosine Similarity**, executing the full diagnostics pipeline and rendering a formatted `Triage_Action_Plan.pdf` inside `LabGym_Diagnostics/` folder.

- **Syntax:**

```
labgym test --config <file.yaml> [inline-overrides]
```

- **Inline Parameters:**

- `--groundtruth_path`: Directory containing true behavior classes
- `--model_path`: Trained model folder path
- `--result_path`: Output statistics/diagnostics folder path
- `--redundant`: List of space-separated true-to-pred error pairs to triage as Redundant (Hypothesis 1)
- `--emergent`: List of error pairs to triage as Emergent Behaviors (Hypothesis 2)
- `--generalization`: List of error pairs to triage as Poor Generalization (Hypothesis 3)

Subcommand `infer`

Headless tracking and behavior categorization on raw input videos or images.

- **Syntax:**

```
labgym infer --config <file.yaml> [inline-overrides]
```

- **Inline Parameters:**

- `--path_to_videos`: Space-separated video files or a folder containing raw files
- `--result_path`: Target results folder
- `--path_to_categorizer`: Trained model path
- `--use_detector`: Toggle detector usage (`true/false`)
- `--path_to_detector`: Object detection model path
- `--t`: Duration offset start (seconds)
- `--duration`: Execution frame window (seconds)

3. YAML Configuration Schema Examples

Below are standard baseline configuration file structures matching each workflow subcommand.

`train_config.yaml` (Categorizer Sub-Network Training)

```
# Categorizer sub-network training
network_type: "combnet"
data_path: "./prepared_dataset"
model_path: "./models/my_new_categorizer"
out_folder: "./cached_frames"

# Architecture hyperparameters
dim_conv: 64
```

```
dim_tconv: 32
channel: 1
time_step: 15
level_conv: 2
level_tconv: 1

# Training parameters
training_onfly: false
augvalid: true
include_bodyparts: true
std: 0.0
background_free: true
black_background: true
behavior_mode: 0
social_distance: 0.0
```

train_detector_config.yaml (Detector Model Training)

```
# Detector training
network_type: "detector"
model_path: "./models/my_new_detector"

# Dataset paths
path_to_annotation: "./dataset/annotations.json"
path_to_trainingimages: "./dataset/images"

# Hyperparameters
iteration_num: 1000
inference_size: 480
```

test_config.yaml (Diagnostics & Triage Plan Generation)

```
# Headless testing parameters
groundtruth_path: "./ground_truth_examples"
model_path: "./models/my_new_categorizer"
result_path: "./evaluation_results"

# Triage Hypothesis mapping (Optional)
# If left blank, all systematic major confusions automatically calculate
# centroids and embed their reference captures directly.
redundant:
  - "behavior_jump -> behavior_leap"
emergent:
  - "behavior_groom -> behavior_scratch"
generalization:
  - "behavior_walk -> behavior_run"
```

infer_config.yaml (Behavior Tracking & Prediction)

```
# Batch inference setup
path_to_videos:
  - "./raw_videos/video_batch_1"
result_path: "./output_results"
path_to_categorizer: "./models/my_new_categorizer"

# Object Detection configs
use_detector: true
path_to_detector: "./models/my_new_detector"
detector_batch: 4
animal_kinds:
  - "mouse"

# Video properties decoding & constraints
decode_animalnumber: true
stable_illumination: true
t: 0.0
duration: 60.0
```

4. SLURM SBatch Execution Example

Below is a baseline script configuration for submitting parallel parameters sweeping runs on SLURM clusters.

```
#!/bin/bash
#SBATCH --job-name=labgym_sweep
#SBATCH --output=logs/labgym_%A_%a.out
#SBATCH --error=logs/labgym_%A_%a.err
#SBATCH --array=1-3
#SBATCH --time=02:00:00
#SBATCH --partition=gpu
#SBATCH --gres=gpu:1

# Load modules / Activate virtual env
module load python/3.10
source venv/bin/activate

# Define configurations list for array jobs
declare -A RATES
RATES[1]=0.001
RATES[2]=0.0005
RATES[3]=0.0001

LR=${RATES[$SLURM_ARRAY_TASK_ID]}

# Execute using baseline config and inline parameter overrides
labgym train \
  --config baseline_train_config.yaml \
  --model_path "./models/model_sweep_${SLURM_ARRAY_TASK_ID}" \
  --learning_rate $LR
```

